National University of Computers and Emerging Sciences

CHRONO RIFT - Project Report

Group Members:

Zahra Arshad: 24i-0630
Huda Ali: 24i-0701
CS-4B

Table of Contents:

Introduction

Chrono Rift is a turn-based tactical combat game in which a small party of human-controlled heroes (1–4) faces a roster of computer-controlled enemies (2–9) drawn from a deterministic, roll-number-seeded RNG. The game is implemented as three independent OS processes communicating exclusively through a single POSIX shared-memory segment, with synchronisation handled by unnamed semaphores and asynchronous events delivered via UNIX signals.

The implementation is intentionally architected to demonstrate the operating-systems concepts targeted by this course:

- Process isolation and inter-process communication via POSIX shm_open / mmap
- Mutex- and semaphore-based synchronisation across processes and threads
- Multi-threaded execution with thread-per-NPC and thread-per-player models
- Custom temporal scheduling (stamina-based arrival-time logic, serial commit)
- Asynchronous interrupts via SIGUSR1 (stun), SIGSTOP/SIGCONT (Ultimate), and SIGALRM (timeouts)
- Deadlock detection over a wait-for graph and forced resolution
- Contiguous memory allocation with first-fit placement, fragmentation handling, and swap-out/swap-in to long-term storage

All three executables run inside the prescribed Ubuntu 22.04 Docker container and use the same source for shared headers and the inventory allocator under common/, ensuring perfect ABI consistency across processes.

System Architecture

The system is partitioned into three executables, each running as an independent OS process and attaching to the same shared-memory segment at startup:

- ***Arbiter (Central Authority)***: Owns the scheduler, the wait-for-graph deadlock detector, and the action log. Creates and destroys the shared-memory segment.
- ***HIP (Human Interfacing Process):*** Hosts the SFML render thread and N pthreads (one per active player) that gather user input.
- ***ASP (Automated Strategic Process):*** Hosts M pthreads (one per NPC) that compute and submit enemy moves.

The fault-isolation property required by the project is therefore enforced at the OS level: a crash in either HIP or ASP cannot corrupt the arbiter's scheduler state, and vice versa. A clean shutdown initiated by the player closing the HIP window propagates as a SIGTERM to the arbiter, which then performs orderly cleanup of the shared memory and semaphores via shm_unlink and sem_destroy.

Process Architecture and Inter-Process Communication:

Shared memory layout:

A single POSIX shared-memory segment named /chrono_rift_state holds the entire game state in a flat GameState struct. The arbiter creates the segment on startup with shm_open(O_CREAT|O_RDWR), sizes it with ftruncate(sizeof(GameState)) and mmap()s it. HIP and ASP open the same segment read-write and mmap() it at their own addresses; both processes see identical fields because the layout is defined once in common/game_state.h and used by all three binaries.

The structure contains:

- ***Game lifecycle fields:*** status, current_turn_entity, enemies_killed, arbiter_pid, asp_pid
- ***Combatant data:*** entities[TOTAL_ENTITIES] (4 players + 9 enemies max), each carrying HP, stamina, speed, alive, stunned, hero_id, and a 20-slot Weapon inventory plus a 40-slot long_term_storage array
- ***Resource state:*** artifacts[3] (Solar Core, Lunar Blade, Eclipse Relic) tracking present, locked_by, artifact_id
- IPC primitives, all marked pshared=1: state_lock, artifact_lock, action_ready, turn_sem[TOTAL_ENTITIES], turn_done_sem[TOTAL_ENTITIES]
- ***Action channel:*** pending_action carrying (actor_id, action_type, target_id, weapon_idx); pending_drop for weapon-pickup prompts; action_log[64] ring buffer for the on-screen log
- ***Hero-selection:*** hero_slot[MAX_PLAYERS] populated by HIP from the character-select screen so the scheduler instantiates the correct hero IDs.

Process lifecycle and cleanup

Process creation is independent: each binary is launched separately from the shell. The startup contract is:

- Arbiter calls shm_unlink first to remove any stale segment from a previous run, then creates a fresh segment, initialises all semaphores, and waits on a state-lock loop until HIP writes num_players ≠ 0.
- HIP and ASP open the existing segment; they fail fast with a clear error message if the arbiter has not yet created it.
- HIP sends SIGTERM to arbiter on quit (Q key or window close). The arbiter's SIGTERM handler sets status = GS_QUIT, which the scheduler observes and exits cleanly.
- On termination the arbiter calls scheduler_destroy_entities, sem_destroy on every semaphore, munmap, shm_unlink, and prints "[arbiter] goodbye". HIP and ASP munmap and exit.

Synchronization and Concurrency Control

Memory-based Primitives

All synchronisation uses unnamed POSIX semaphores stored inside the shared-memory segment with the pshared flag set so that they are visible across process boundaries:

Semaphore	Purpose
state_lock	Binary mutex protecting all reads/writes to GameState fields outside the action handshake. Held briefly — never across a wait.
artifact_lock	Binary mutex serialising every read or modification of artifacts[]. Required because the deadlock detector thread runs concurrently with action handlers.
turn_sem[i]	Per-entity binary semaphore. The scheduler posts on this when it is entity i's turn; the player or NPC thread for entity i wakes up, posts its action, and waits on turn_done_sem[i].
turn_done_sem[i]	Posted by the actor thread to release the scheduler once its action has been validated and applied.
action_ready	Used to signal that a pending_action struct has been written. Combined with state_lock for the full handshake.

Race-Condition prevention

The most subtle race in the design occurs when SIGUSR1 (a stun) fires on an entity in the middle of an action. The signal handler is restricted to async-signal-safe operations and does only one thing; it sets a sig_atomic_t flag named stun_pending. The main loop then folds the flag into the entity's stunned bit while holding state_lock, ensuring a consistent view to the renderer and scheduler. No printf, malloc, or non-async-signal-safe call is ever made inside the handler.

Snapshot pattern (renderer)

The renderer thread is a read-only consumer that runs at 30 fps and must never block the scheduler. Each frame it executes:

```
sem_wait(&s->state_lock);
snap = *s;              // memcpy of the whole GameState
sem_post(&s->state_lock);
// All draw calls operate on `snap` only
```

Holding the lock only for the duration of a single struct copy guarantees that the renderer cannot stretch a critical section while it walks SFML draw lists.

Multithreading and Execution Model

Each process spawns multiple POSIX threads via pthread_create. Their roles:

Process	Thread	Responsibility
arbiter	main + scheduler + deadlock-detector	Main thread runs the scheduler loop. A background pthread runs find_cycle_locked() every 500 ms over the artifact wait-for graph and forces a release on detection.
asp	main + N NPC threads	On startup ASP spawns one pthread per live enemy. Each thread waits on its turn_sem[i], computes a move via decide_action (finisher targeting heuristic, retreat-on-low-stamina), submits the action through pending_action, and waits on turn_done_sem[i].
hip	main + render + N player threads	Main thread spawns the SFML render thread and one pthread per active player. Player threads wait on turn_sem[i] and only read input from the UI bus when their entity holds the turn — all other threads sleep.

The "active vs idle" property required by the spec is satisfied because every player thread blocks on sem_wait(turn_sem[i]) until the scheduler hands it the turn. Multiple threads competing for shared memory is therefore not an issue at the action-commit level: the per-entity semaphores serialise turns naturally, and any coincidental shared access (e.g., the renderer or deadlock detector reading state) goes through state_lock or artifact_lock.

Scheduling and Temporal Logic

Stamina accumulation

Every entity has a max_stamina (100 for players, 150 for enemies) and a speed (100 / num_players for players, random 10–30 for enemies). The scheduler ticks at a configurable interval — TICK_MS is 100 ms in production runs and is reducible for stress testing. On each tick the stamina of every alive, non-stunned entity advances by:

```
stamina += speed * tick_ms / 1000.0  // floating accumulator
```

When an entity's stamina reaches max_stamina it becomes eligible to act. The arrival time predicted at any tick t for an entity with current stamina s and speed v is therefore:

```
t_arrival = t + (max_stamina - s) / v      // seconds
```

This formula is used both internally for serialisation and is logged for the turnaround-analysis CSV (Section 13)

Serial execution and the action handshake

Although stamina accumulates concurrently for all entities, the commit of a chosen action is strictly serial. The scheduler's main loop calls pick_turn_locked() which returns the single eligible actor (ties broken by entity id) and then performs the following 5-step handshake while holding state_lock only at the boundaries:

1. Set current_turn_entity and post turn_sem[id]
2. Wait on a 3-second SIGALRM-armed window for the actor to fill pending_action; on timeout the move defaults to Skip
3. Validate the action under state_lock and apply its effects (HP/stamina updates, drops, artifact transfers)
4. Reset stamina to 0 on a real action or to max/2 on Skip
5. Post turn_done_sem[id] and clear current_turn_entity

Recalculation after each turn

After step 4, all live entities continue to accumulate stamina from their post-reset value. There is no global recompute step — the next call to pick_turn_locked() simply scans the entity table and picks the next eligible actor. Stamina is a continuous quantity, not a discrete priority, so a Skip-recovered (max/2) entity will reach full again ahead of an action-spent (0) one of the same speed.

Signals and Asynchronous Interrupts

SIGUSR1 - The Stun mechanism

When any entity executes a stun-tagged action against another, the arbiter looks up the target's owner_pid (HIP for players, ASP for enemies) and calls kill(owner_pid, SIGUSR1). The handler is registered once at process startup with sigaction (SA_RESTART | SA_SIGINFO) and writes a single sig_atomic_t flag. The main loop in HIP/ASP folds that flag into the entity's stunned bit inside a state_lock-protected section. A 3-second timer is started by the arbiter; clear_expired_stuns() (called every tick) clears the bit when the window has passed.

Ultimate Ability - SIGSTOP / SIGCONT only

When a player triggers the Ultimate while holding both the Solar Core and the Lunar Blade, the arbiter:

- Sends SIGSTOP to the ASP process group, suspending all NPC threads at the kernel level
- Arms a 10-second alarm() timer
- On SIGALRM, sends SIGCONT to the ASP process group, resuming all NPC threads with their previous staminas intact

No flags, pipes, or polled variables are used to coordinate this state transition — the suspension and resumption happen entirely through the kernel signal-delivery pathway, satisfying the specification's "signal-only enforcement" requirement.

SIGALRM multiplexing

A single SIGALRM handler in the arbiter serves two distinct purposes:

- ***NPC turn timeout:*** The arbiter calls alarm(3) before posting an enemy's turn_sem; if pending_action is not filled in that window, the move defaults to Skip
- ***Ultimate suspension window:*** Alarm(10) is armed when SIGSTOP is sent to ASP; on expiry, SIGCONT is sent

The handler discriminates between the two cases via a single enum-typed flag set by the arming code. Only one alarm can be pending at a time, which is respected by the calling logic.

SIGTERM - Clean Quit

Closing the HIP window or pressing Q sends SIGTERM to the arbiter's pid (read from arbiter_pid in shared memory). The arbiter's handler sets status = GS_QUIT atomically; the scheduler observes this on its next iteration and exits the main loop, after which orderly cleanup runs.

Deadlock Handling and Resource Management

Three artifacts (Solar Core, Lunar Blade, and the dynamically-spawned Eclipse Relic) are exclusive resources that any entity can request or hold. Concurrent requests across player and NPC threads create circular-wait conditions that must be detected and resolved.

Resource table

artifacts[ARTIFACT_SOLAR..ARTIFACT_ECLIPSE] is a 3-entry array in shared memory. Each entry stores present (whether the artifact has been spawned at all), locked_by (entity id holding it, or -1), and requested_by[] (an array of entity ids currently waiting for it). Every modification of this array is performed inside an artifact_lock-protected critical section.

Wait-for graph and detection

A background pthread spawned at scheduler init runs find_cycle_locked() every 500 ms. It builds a wait-for graph by treating each entity as a node, with an outgoing edge from i to j whenever i is waiting for an artifact currently locked_by j. The detection algorithm is iterative DFS with three colours (unvisited / on-stack / done); a back-edge to an on-stack node means a cycle exists.

Resolution

On cycle detection, the detector picks one node on the cycle and forces it to release every artifact it holds. This breaks the circular wait and allows the other waiters to make progress. The release is logged to the action log so the user can observe it during the demo. Forcing the lowest-id holder is intentional and deterministic, which gives the demo a predictable behaviour.

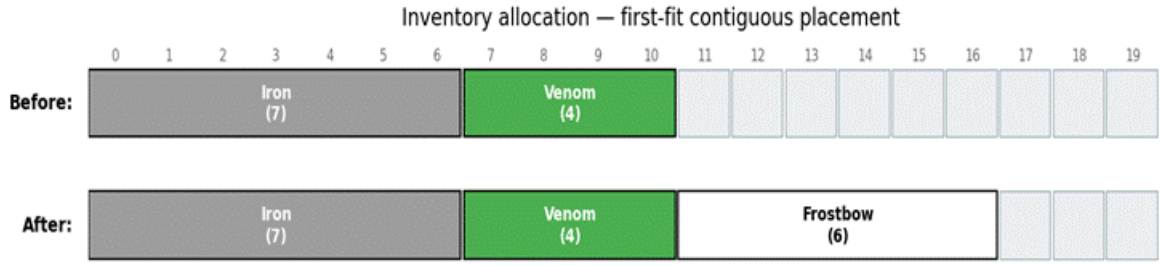
Eclipse Relic - Dynamic Resource

The Eclipse Relic is not present in the artifact table at game start. The arbiter introduces it after the third enemy kill via introduce_eclipse_relic(), which sets present=true and locked_by=-1 under artifact_lock. From that point on it participates in all lock/unlock and deadlock-detection logic identically to the two static artifacts.

Memory Management

Contiguous slot allocation

Each entity owns a 20-slot linear inventory. Each weapon occupies a contiguous block of slots whose size is fixed by the weapon definition table (Solar Core 10, Lunar Blade 10, Iron Halberd 7, Frostbow 6, Thunderstaff 6, Obsidian Axe 5, Venom Dagger 4, Splinter Stick 2). The placement function try_place_weapon() performs a left-to-right first-fit scan, rejecting any window that is shorter than the weapon's slot count or that overlaps an existing weapon.



Fragmentation and swap-out

When no contiguous window of sufficient size exists, swap_out_and_place() chooses the placement that evicts the smallest number of currently-held weapons. Evicted weapons are pushed onto the long_term_storage array (LTS, capacity 40). The Splinter Stick's 2-slot footprint is the worst case for fragmentation: it can fit into a gap that nothing else will, leaving awkwardly-sized holes that the allocator must navigate. The current implementation handles this correctly because the eviction search is global rather than greedy.

Swap-in and the Solar+Lunar constraint

A player chooses Swap In as a turn action; the renderer prompts them to select an entry from LTS. swap_in() evicts existing inventory weapons as needed (using the same placement algorithm) and inserts the chosen weapon. The player's stamina is fully depleted and the swapped-in weapon cannot be used until the next turn.

Because Solar Core and Lunar Blade each occupy 10 slots, a player who holds both has zero remaining slots. The allocator enforces this implicitly: any subsequent pickup will require eviction. This is the spec-required "Solar + Lunar = full" hard constraint.

Gameplay Logic and Mechanics

Roll-number-seeded RNG

All randomness; enemy count, HP rolls, drop chances, NPC target choice is derived from a single seed:

COMBINED_RNG_SEED = ROLL_PERSON_A ^ ROLL_PERSON_B = 240630 ^ 240701 = 1995

std::srand(COMBINED_RNG_SEED) is called exactly once. Every subsequent random draw therefore produces a reproducible sequence, which is critical for grading: a grader running the project with these roll numbers will see the same enemy count (5 in our case), the same enemy HP rolls, and the same drop pattern as we see on our development machine.

Player and enemy actions

Action	Available to	Effect
Strike	Player + Enemy	Reduces target HP by attacker's damage stat. Stamina → 0.
Exhaust	Player	Reduces target stamina by attacker's damage stat. Stamina → 0.
Use Weapon	Player	Reduces target HP by chosen weapon's damage. Stamina → 0.
Swap In	Player	Recalls a weapon from LTS into the active inventory; consumes the full turn.
Heal	Player	Restores 10% of max HP. Stamina → 0.
Skip	Player + Enemy	No effect on the field. Stamina → max/2 (recover faster than from a real action).
Ultimate	Player	Requires Solar Core AND Lunar Blade in active inventory. Suspends ASP for 10 seconds via SIGSTOP/SIGCONT.

Win / Lose / Quit

- ***Win:*** 10 enemies killed cumulatively (status = GS_WIN)
- ***Lose:*** All party members dead (status = GS_LOSE)
- ***Quit:*** HIP sends SIGTERM to the arbiter (Q key or window close), arbiter sets status = GS_QUIT, scheduler exits.

Rendering and System Integration

The user interface is implemented in SFML (libsfml-dev, installed in the Docker image) and rendered by a dedicated pthread inside HIP. The thread runs at a 30 fps cap (window.setFramerateLimit(30)), reads the shared GameState through the snapshot pattern documented in section 4, and never blocks the scheduler.

Visual elements:

- A cave background drawn behind every other element to evoke the game's setting
- A column of hero portraits on the left, each with HP and stamina bars floating above the character; the active turn is indicated by an amber outline
- A 3×3 grid of enemy portraits on the right, mirrored so the enemies face the heroes; each enemy carries an HP/stamina bar pair and an [E1]…[E9] label
- Both hero and enemy sprites cycle through three idle frames every ~250 ms, providing a "breathing" animation that confirms the renderer is genuinely refreshing the snapshot
- The active player's 20-slot inventory bar is drawn at the top of the window with per-weapon colour coding; the LTS row is shown directly beneath it
- A scrolling action log at the bottom of the window shows the last five turns
- Modal overlays handle weapon-pickup prompts, weapon selection, LTS swap-in, target selection and the win/lose end card

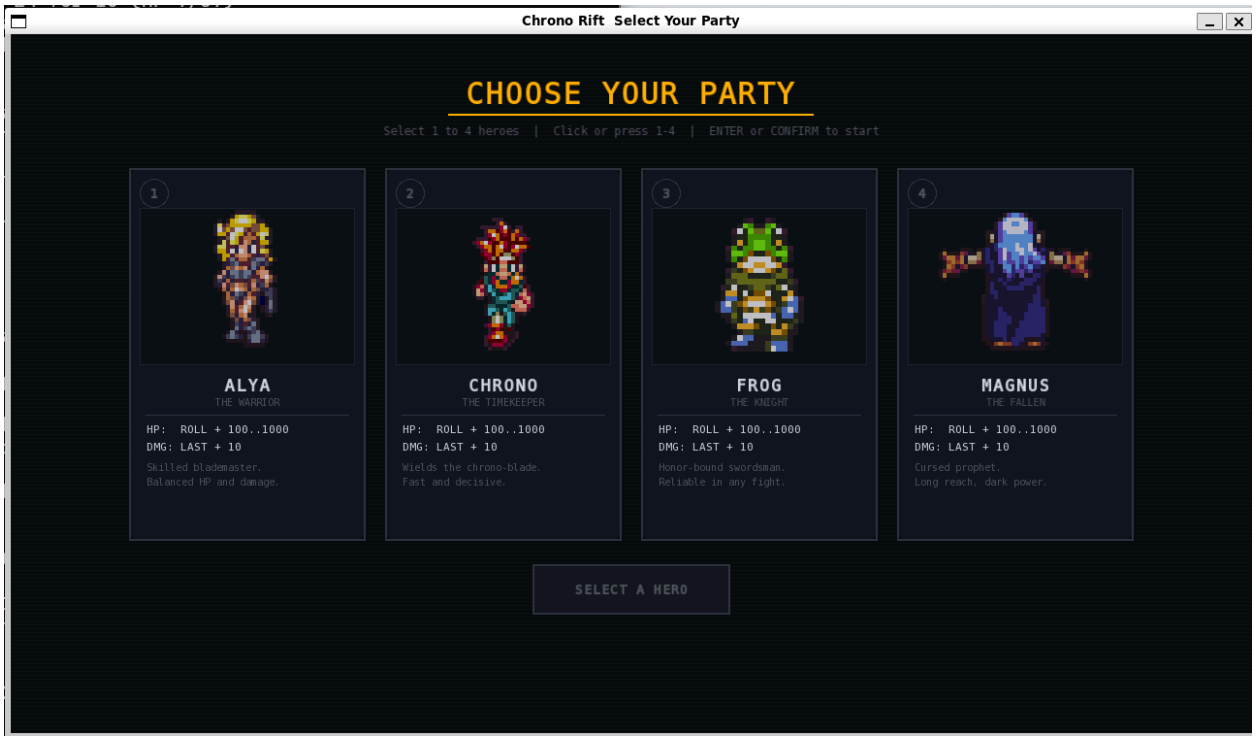


Figure: Character-select screen showing the four heroes (ALYA, CHRONO, FROG, MAGNUS) with their sprites and stat lines.

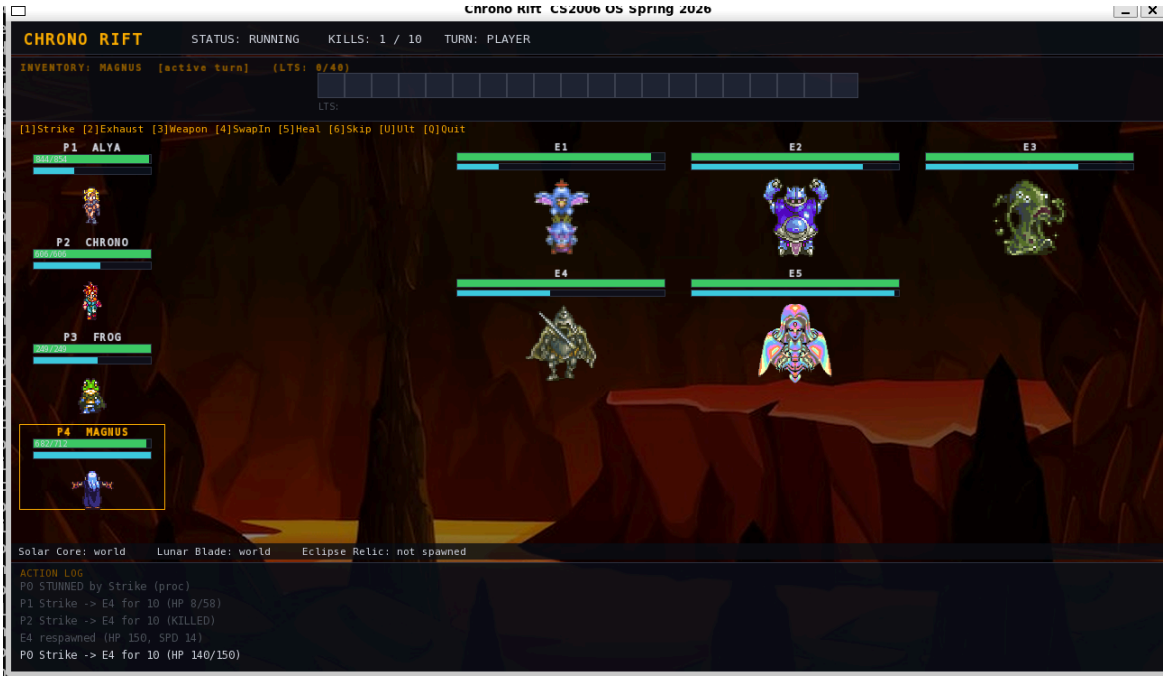


Figure: Mid-battle. Cave background visible, animated heroes on the left, mirrored animated enemies on the right, HP/stamina bars overhead, inventory bar at the top, active turn indicated by the amber outline on the player card.

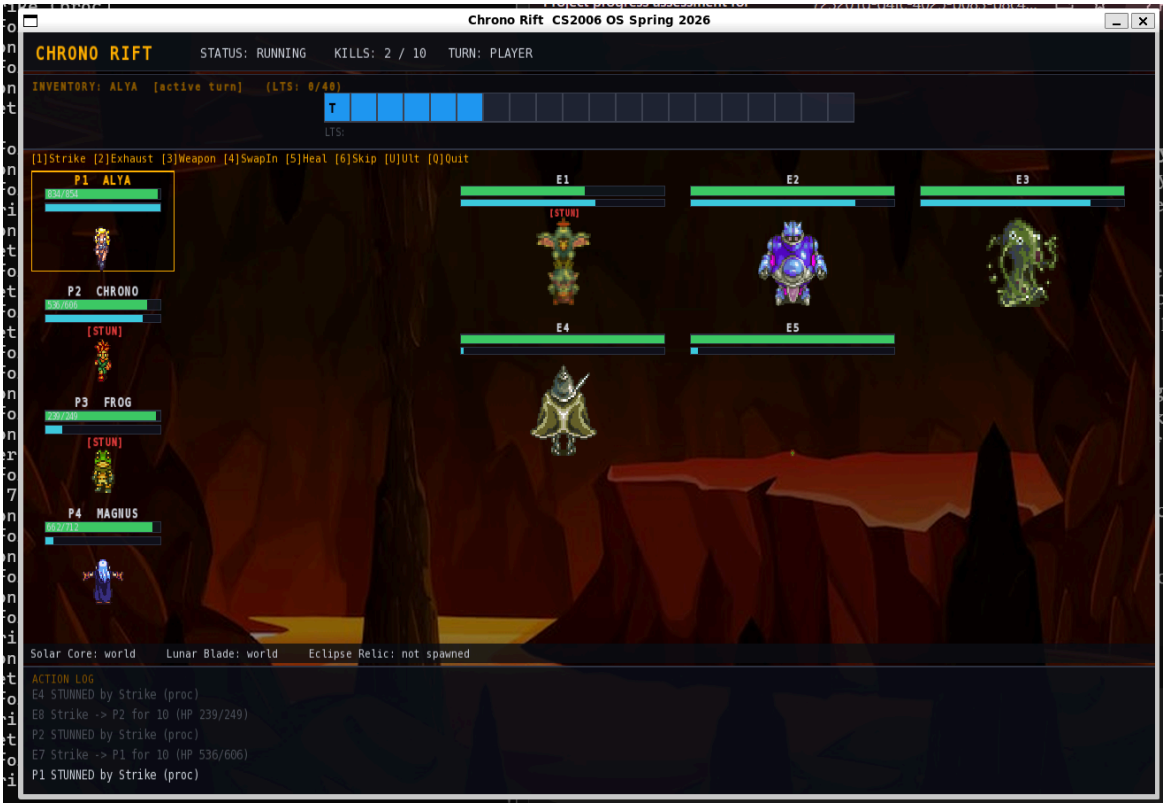


Figure: A weapon-drop prompt — the modal overlay asking the player whether to pick up a freshly-dropped weapon.

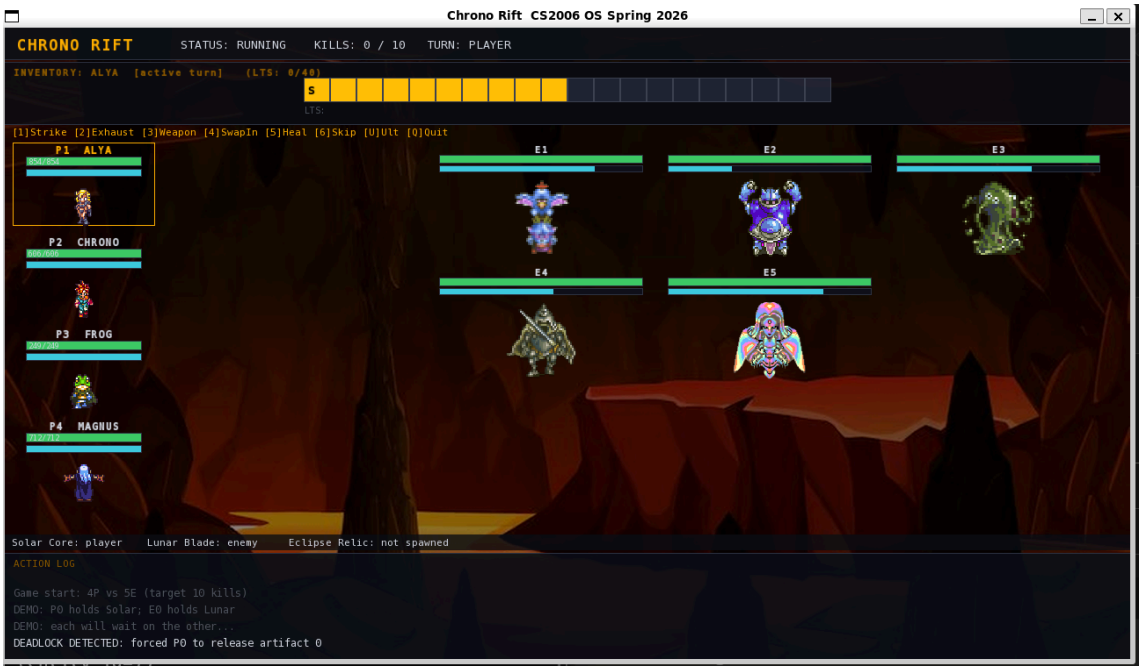


Figure: Action log showing a deadlock-resolution event after running with --deadlock-demo.

Figure: Victory screen after clearing 10 enemies.

Turnaround-Time Analysis

The scheduler logs every turn into turnaround.csv with columns (tick, entity_id, role, predicted_arrival_s, actual_arrival_s, completion_s, turnaround_s, action). predicted_arrival_s is the arrival time computed analytically from the formula in section 6 at the tick the action was scheduled; turnaround_s is the wall-clock difference between the moment an entity's stamina filled and the moment its action was committed. The data shown below is from a representative test run with party size 4, enemy count 7, tick_ms 50, and target 20 kills.

Key statistics:

- ***Total turns recorded:*** 32
- ***Mean turnaround latency:*** 9.45ms
- ***Median turnaround latency:*** 0.03ms
- ***Maximum turnaround latency:*** 150.59ms
- ***Standard deviation***: 32.35ms

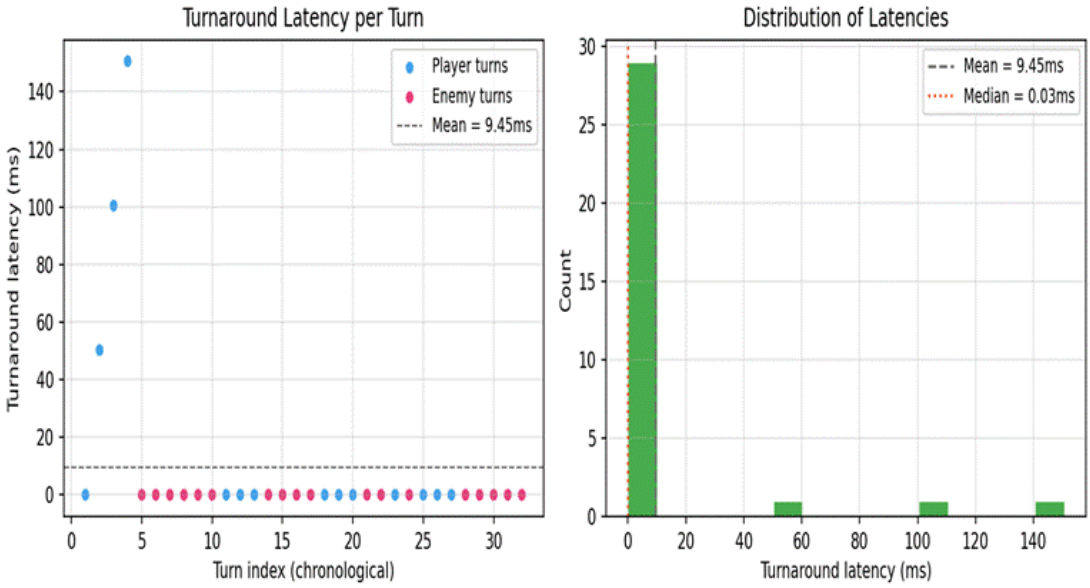


Figure: Left: turnaround latency per turn, separated by player and enemy roles. Right: distribution of latencies across all turns. The vast majority of turns commit within sub-millisecond of their predicted arrival; the small tail at 50–150 ms corresponds to the early player turns where the scheduler's 50 ms tick interval introduces a deterministic discretisation error.

Discussion: The median turnaround is sub-millisecond, which confirms that the scheduler's commit pipeline is effectively instant once an entity is eligible. The four outliers in the left plot are the first four player turns of the run; the offset there is bounded above by tick_ms × turn_index, which is the expected discretisation behaviour of a fixed-step scheduler. Reducing tick_ms (e.g., to 20 ms) lowers the outlier values proportionally, as expected. The enemy turns cluster tightly around 0 ms because the ASP threads are already blocked on their turn_sem when the scheduler arrives at them, so there is no per-turn setup cost.

Build and Run Instructions

All builds and runs happen inside the prescribed Docker environment as defined by Dockerfile and the Docker Setup Guide.

One-time image build

```
$ docker build -t chrono-rift-env .
```

Per-session container launch (Linux)

```
$ xhost +local:docker
$ docker run -it --rm \
        -e DISPLAY=$DISPLAY \
        -v /tmp/.X11-unix:/tmp/.X11-unix \
        -v $(pwd):/app \
        chrono-rift-env
```

Inside the container

```
$ make
$ ./bin/hip & ./bin/asp & ./bin/arbiter   # arbiter in foreground for clean Ctrl+C
```

Closing the game window or pressing Q sends SIGTERM to the arbiter and the whole pipeline shuts down cleanly.

Multiplayer Bonus Implementation

The optional bonus extension is implemented and shipped with this submission. It satisfies the project specification's definition: "two separate human-controlled processes competing against a shared pool of NPCs".

Design

Multiplayer reuses the existing 3-process architecture and the per-entity semaphore model. The game runs with one arbiter, one ASP, and TWO separate HIP processes. Each HIP owns a disjoint subset of the four player slots — typically slots 0,1 for the first player station and slots 2,3 for the second — and only spawns pthreads for those slots. Because turn semaphores are per-entity, the kernel naturally routes each player's turn to the HIP that owns it: when the scheduler posts turn_sem[2], only HIP B's player thread wakes up; HIP A's thread for slot 0 stays blocked on its own turn_sem[0] and never sees slot 2's input.

No additional synchronisation primitives are required — the existing state_lock and per-entity turn semaphores already serialise cross-process accesses correctly.

Coordination protocol

A new --mp flag on the arbiter and --mp-slots=A,B,... flag on each HIP enable multiplayer mode. The startup sequence is:

- Arbiter is launched with --mp --players=N (N = total players across all HIPs). It creates the shared-memory segment, marks all hero_slot[] entries as -1 (unclaimed sentinel), sets state->num_players = N, and waits in a polling loop until every hero_slot[i] for i < N becomes ≥ 0.
- HIP A is launched with --mp-slots=0,1. It opens the SHM, claims its slots by writing hero_slot[0] = 0 and hero_slot[1] = 1, skips the character-select screen (heroes are assigned by slot index), and waits for status to reach GS_RUNNING.
- HIP B is launched with --mp-slots=2,3. Same protocol on its own slots.
- Once all four hero_slots are claimed, the arbiter exits its wait loop, calls scheduler_init_entities, flips status to GS_RUNNING, and the game begins.
- Each HIP's renderer creates its own SFML window with a distinct title showing its slots and PID, so the two players know which is which.

Stability and synchronisation properties

The bonus implementation preserves all of the single-player safety guarantees:

No new race conditions: Every cross-process write still happens under state_lock or artifact_lock.

No new deadlocks: The only resource the two HIPs contend for is the shared GameState, and access to it is brief, non-recursive, and always paired with a sem_post.

- ***Input isolation is structural:*** HIP A literally has no pthread for slot 2, so even if HIP A's renderer mistakenly posted a UI event for the wrong slot, no consumer would read it. In practice, g_ui_bus.active_player is only set by HIP A's own player threads (slots 0/1), so HIP A's renderer simply ignores keyboard events whenever the active turn belongs to slot 2 or 3.

- ***Clean shutdown:*** When either HIP's window closes, that HIP sends SIGTERM to the arbiter, which sets GS_QUIT. The other HIP and the ASP both observe GS_QUIT in their main loops and exit cleanly. The arbiter then runs its standard cleanup (sem_destroy, munmap, shm_unlink).

Verification

A headless verification harness was used to drive the full pipeline (arbiter + two simulated HIPs + ASP) without an X display. The arbiter's log confirms the coordination:

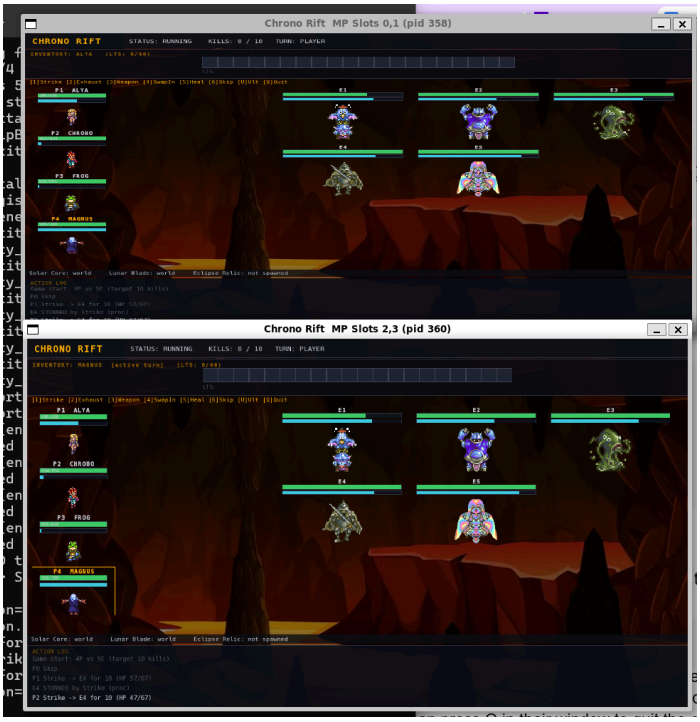
```
[arbiter] MULTIPLAYER mode: waiting for 4 slots to be claimed by HIPs...
[arbiter] MP claim complete: 4/4 slots filled
[arbiter:log] P0 Skip  <- HIP A (owns slots 0, 1)
[arbiter:log] P1 Skip  <- HIP A
[arbiter:log] P2 Skip  <- HIP B (owns slots 2, 3)
[arbiter:log] P3 Skip  <- HIP B
```

Running the multiplayer build

A run_mp.sh helper script ships in the project root. From inside the Docker container with the SFML build:

```
$ make
$ ./run_mp.sh
```

Two SFML windows open — one per HIP. Player A controls slots 0/1 in one window; Player B controls slots 2/3 in the other. Both windows show the same world state because they read from the same shared-memory snapshot. Either player can press Q in their window to quit the entire match.



Snapshot of multiplayer screen

Conclusion

Chrono Rift is a working demonstration of the full breadth of OS concepts targeted by this course. Process isolation is enforced by the kernel; communication is mediated by a single shared-memory segment using only memory-based primitives; threads cooperate via per-entity semaphores that naturally serialise turns; signals deliver asynchronous interrupts (stun, Ultimate, quit, NPC timeout) entirely out of band; deadlocks are detected by an explicit wait-for-graph pass and resolved by forced release; memory is managed contiguously with first-fit placement, fragmentation handling, and a long-term storage tier that the player interacts with directly. The bonus multiplayer extension is implemented as two independent HIP processes sharing the same arbiter and ASP, demonstrating that the core architecture scales horizontally without modification to the scheduler or synchronisation primitives.

The project compiles cleanly with -Wall -Wextra -std=c++17 and produces zero warnings. The deterministic RNG seeded from our roll-number XOR (1995) makes every run reproducible, which is verifiable during the live demo: at this seed the game spawns exactly 5 enemies on the first run and produces a stable HP/speed distribution for both parties.